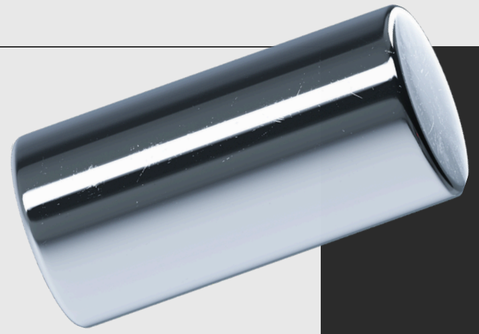
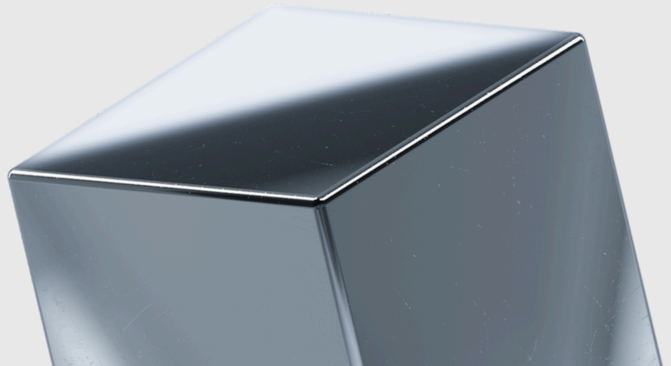
# SHADER PROJECT

A GLSL-BASED 3D RENDERER & EDITOR FOR THE WEB







# MEET THE TEAM



ALI GASIMOV



RASHID HUSEYNOV

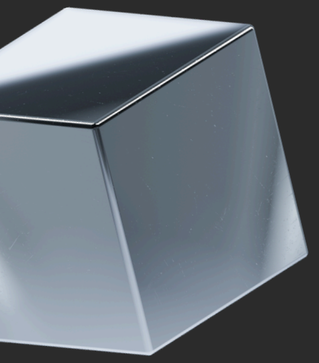


NAZRIN AZIZLI

# TABLE OF CONTENTS


- INTRODUCTION
- WHAT IS WEBGL & THREE.JS?
- PROCEDURAL TERRAIN – FBM
- PROCEDURAL TERRAIN – DOMAIN WARPING & WATER
- HEIGHT-BASED COLORING
- BLINN-PHONG LIGHTING
- POST-PROCESSING PIPELINE
- CAMERA SYSTEM
- SCENE EDITOR
- PROJECT ARCHITECTURE
- SHADER FILES
- TESTING
- RESULTS
- LIMITATIONS
- FUTURE IMPROVEMENTS
- CONCLUSION



# INTRODUCTION



A SHADER IS A SMALL GPU PROGRAM THAT CONTROLS COLOR, LIGHTING, AND EFFECTS OF PIXELS AND VERTICES IN 3D GRAPHICS

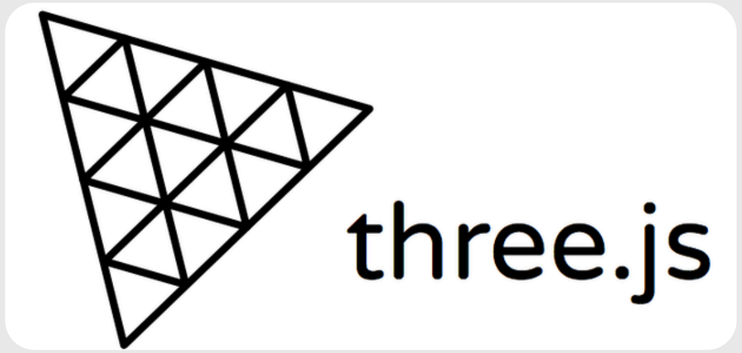
THIS PROJECT BUILDS A COMPLETE REAL-TIME 3D RENDERER THAT RUNS ENTIRELY IN THE BROWSER

ALL VISUAL COMPUTATION IS DONE WITH HAND-WRITTEN GLSL — NO THREE.JS BUILT-IN MATERIALS ARE USED

THE GOAL: TO CREATE A MODULAR AND CONVENIENT INTERFACE FOR WEB DEVELOPERS TO INSERT 3D EFFECTS AND MODELS INTO THEIR APPS

# WHAT IS WEBGL & THREE.JS?




## WEBGL

WebGL exposes the GPU's programmable pipeline to JavaScript, enabling real-time 3D in the browser

## THREEJS

Three.js is a high-level library wrapping WebGL — it handles scenes, cameras, meshes, and lights

**This project uses Three.js's ShaderMaterial to inject raw GLSL, bypassing all built-in shading**

## TWO SHADER TYPES ARE USED

- **VERTEX SHADERS**

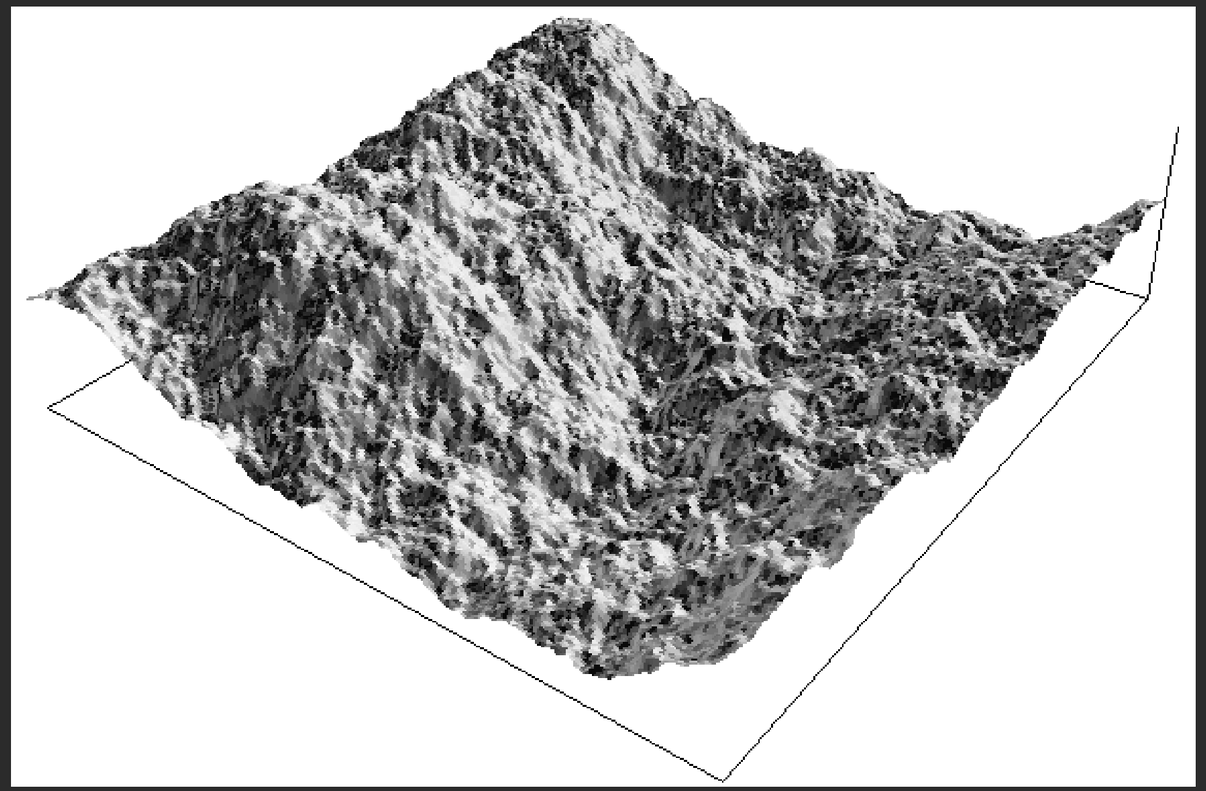
  compute each vertex's final screen position

- **FRAGMENT SHADERS**

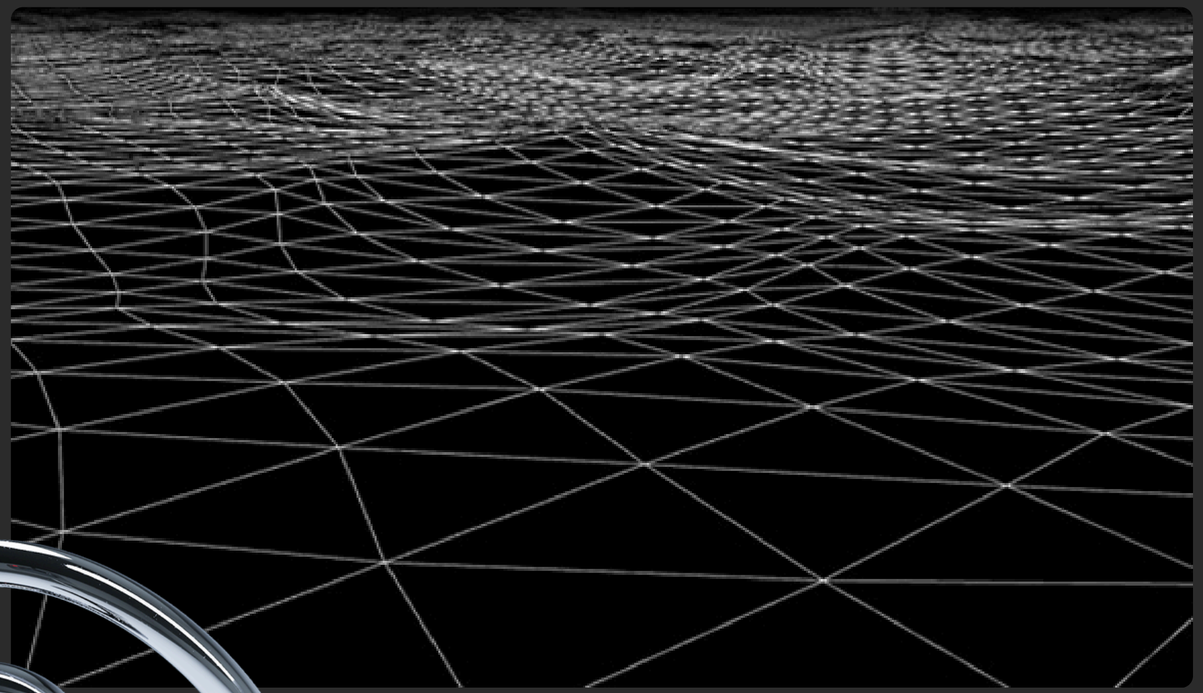
  output the final color for every pixel on screen

# PROCEDURAL TERRAIN: FRACTAL BROWNIAN MOTION

- THE TERRAIN IS A 256×256 VERTEX FLAT PLANE, DISPLACED ON THE GPU NO CPU HEIGHTMAP NEEDED
- HEIGHT IS COMPUTED USING FRACTAL BROWNIAN MOTION (FBM): LAYERING MULTIPLE OCTAVES OF SIMPLEX NOISE
- EACH OCTAVE DOUBLES IN FREQUENCY AND HALVES IN AMPLITUDE
- MORE OCTAVES = MORE FINE SURFACE DETAIL WHILE PRESERVING THE LARGE-SCALE SHAPE
- A RIDGE FBM VARIANT SHARPENS VALLEYS INTO MOUNTAIN RIDGELINES AT ALMOST NO EXTRA COST
- NORMALS ARE APPROXIMATED USING FINITE DIFFERENCES AND PASSED TO THE FRAGMENT SHADER FOR LIGHTING



# PROCEDURAL TERRAIN: DOMAIN WARPING & WATER



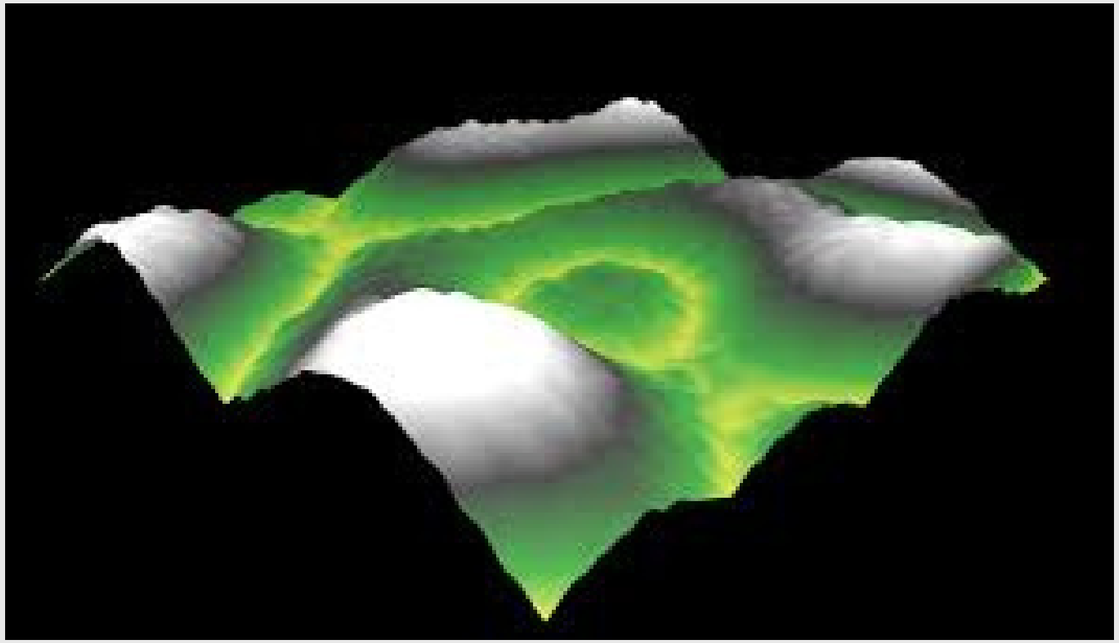
- DOMAIN WARPING OFFSETS INPUT COORDINATES USING A SECOND NOISE EVALUATION BEFORE THE MAIN FBM PASS
- THIS PRODUCES FLOWING, FOLDED STRUCTURES REMINISCENT OF REAL ERODED TERRAIN AND RIVER VALLEYS
- A SINE-WAVE BLEND MODE SIMULATES ANIMATED WATER SURFACES USING THREE OVERLAPPING SINE WAVES WITH DIFFERENT DIRECTIONS, FREQUENCIES, AND SPEEDS
- THE BLEND BETWEEN LAND AND WATER IS CONTROLLED BY A SINGLE UNIFORM ADJUSTABLE AT RUNTIME
- BOTH MODES SHARE THE SAME NORMAL COMPUTATION, SO LIGHTING STAYS CONSISTENT ACROSS ALL TERRAIN TYPES

- THE FRAGMENT SHADER ASSIGNS COLOR BASED ON EACH VERTEX'S HEIGHT VALUE PASSED FROM THE VERTEX SHADER

- SEVEN COLOR ZONES FROM DEEP OCEAN TO SNOW CAPS:

  Deep Water · Shallow Water · Sand / Beach · Lowland Grass · Highland · Bare Rock · Snow Cap

# HEIGHT-BASED COLORING & SURFACE DETAIL



- ADJACENT ZONES BLEND SMOOTHLY USING LINEAR INTERPOLATION: NO HARD EDGES

- HIGH-FREQUENCY SIMPLEX NOISE ADDS MICRO-DETAIL ON TOP: GRASS TEXTURE, ROCK GRAIN

- EXPONENTIAL FOG IS APPLIED BASED ON CAMERA DISTANCE, ADDING NATURAL DEPTH TO THE SCENE

# BLINN-PHONG LIGHTING MODEL

LIGHTING IS SPLIT INTO THREE ADDITIVE TERMS:

**AMBIENT**

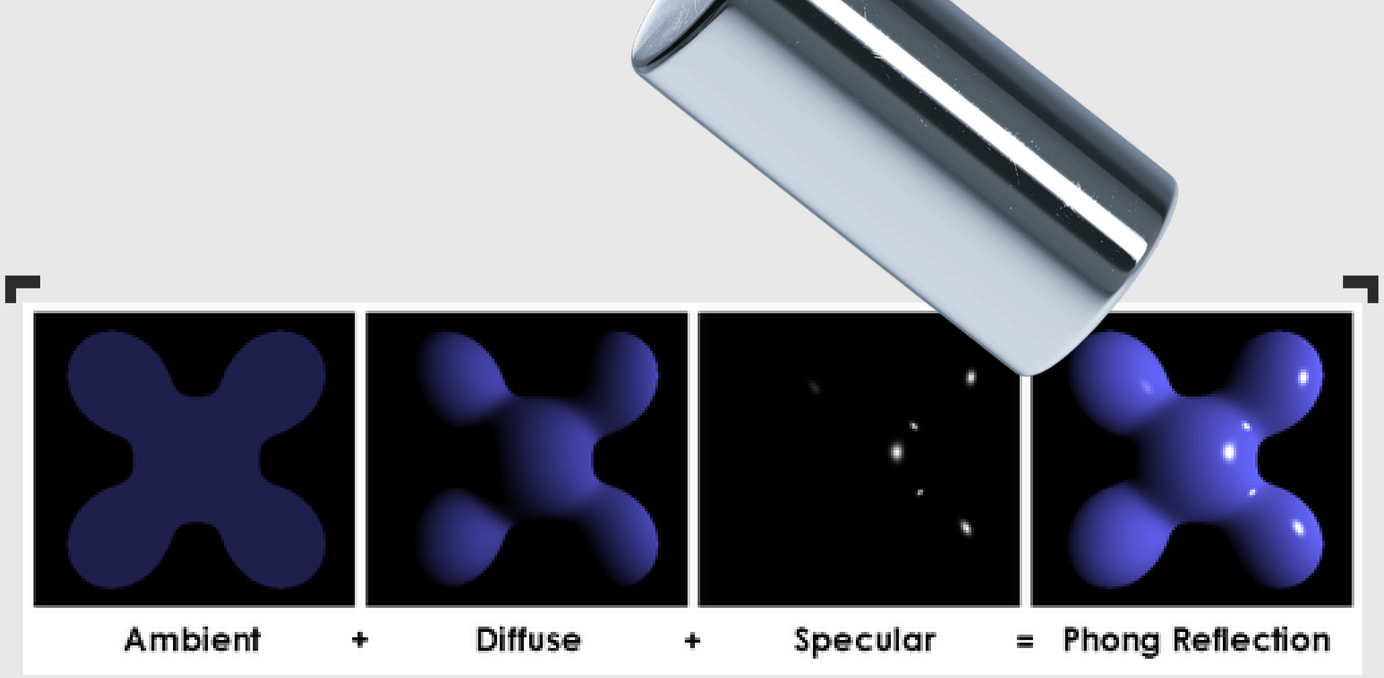
constant indirect light, prevents fully black shadows

**DIFFUSE**

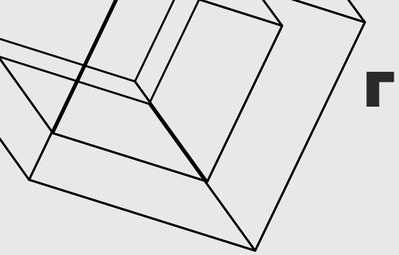
light scattered from rough surfaces following Lambert's cosine law

**SPECULAR**

mirror-like highlights using the half-vector between the light and view direction




- All calculations done per-pixel in the fragment shader using world-space positions and normals
- A debug mode lets you isolate each component at runtime using keys 1 through 4



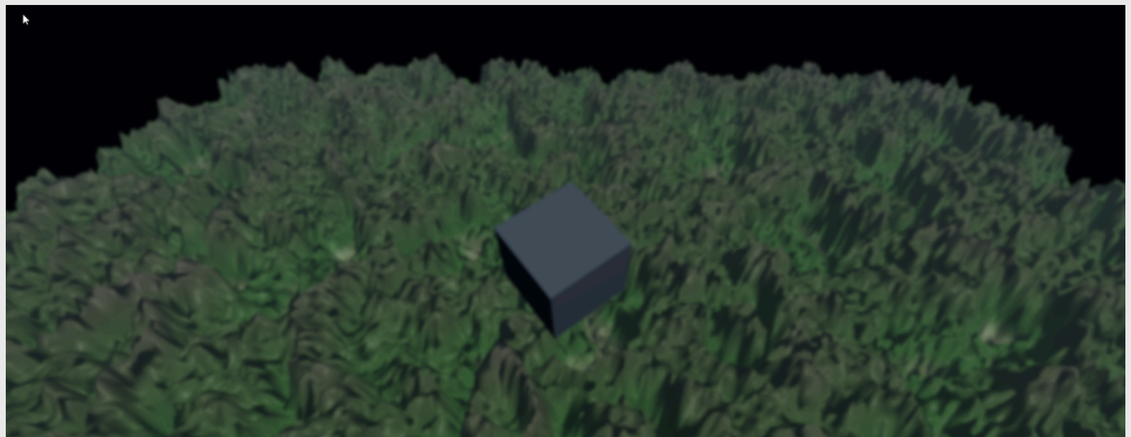
# POST-PROCESSING PIPELINE

THE SCENE RENDERS INTO AN OFF-SCREEN BUFFER FIRST — NOT DIRECTLY TO THE CANVAS

THAT TEXTURE PASSES THROUGH A CHAIN OF FULL-SCREEN SHADER PASSES BEFORE REACHING THE SCREEN

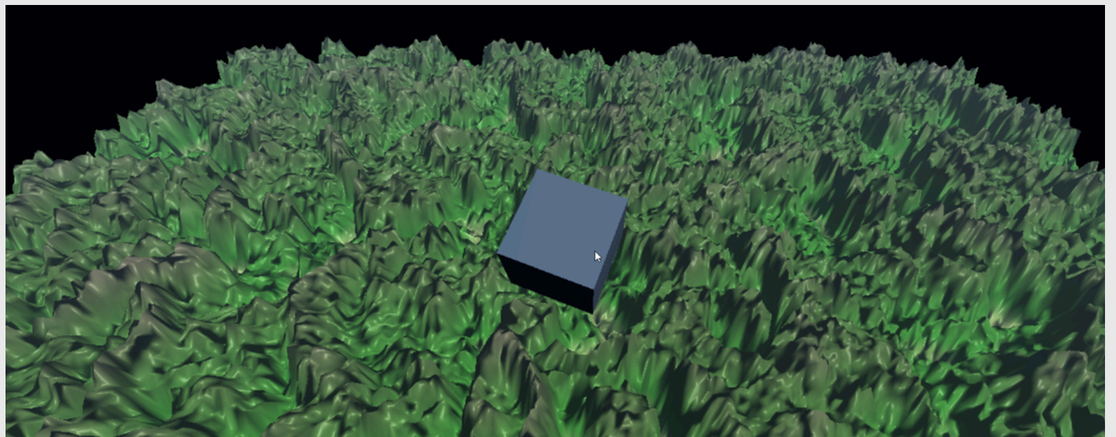
**GAUSSIAN BLUR (TWO-PASS SEPARABLE KERNEL)**

- O(k) texture samples per pass instead of $O(k^2)$ for a naive 2D kernel
- Multiple iterations increase blur radius without widening the kernel

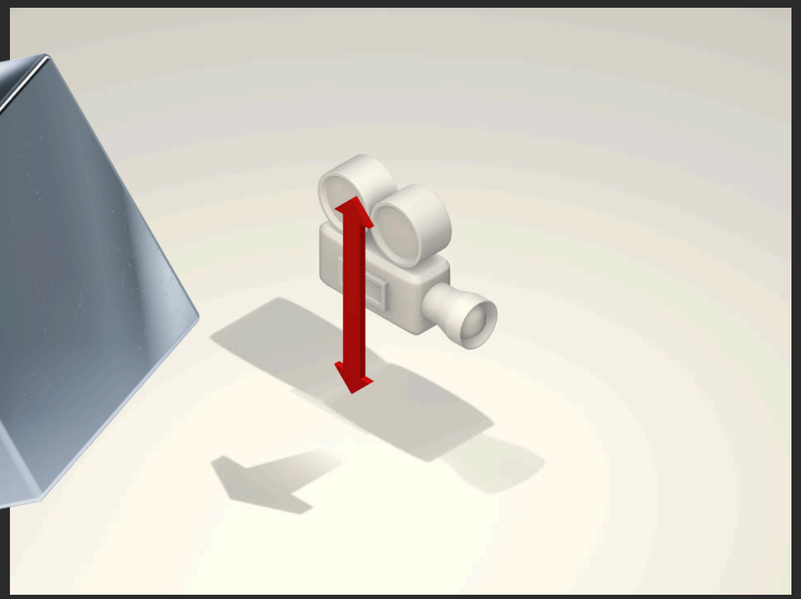


**BLOOM**

- Extract pixels above a luminance threshold
- Blur the extracted bright image
- Composite back onto the scene with Reinhard tone mapping to prevent overexposure



# CAMERA SYSTEM



USES THE BROWSER'S POINTER LOCK API FOR SMOOTH FIRST-PERSON MOUSE LOOK

**MOVEMENT CONTROLS:**

- WASD — horizontal movement
- Q / E — vertical movement
- Shift — sprint multiplier

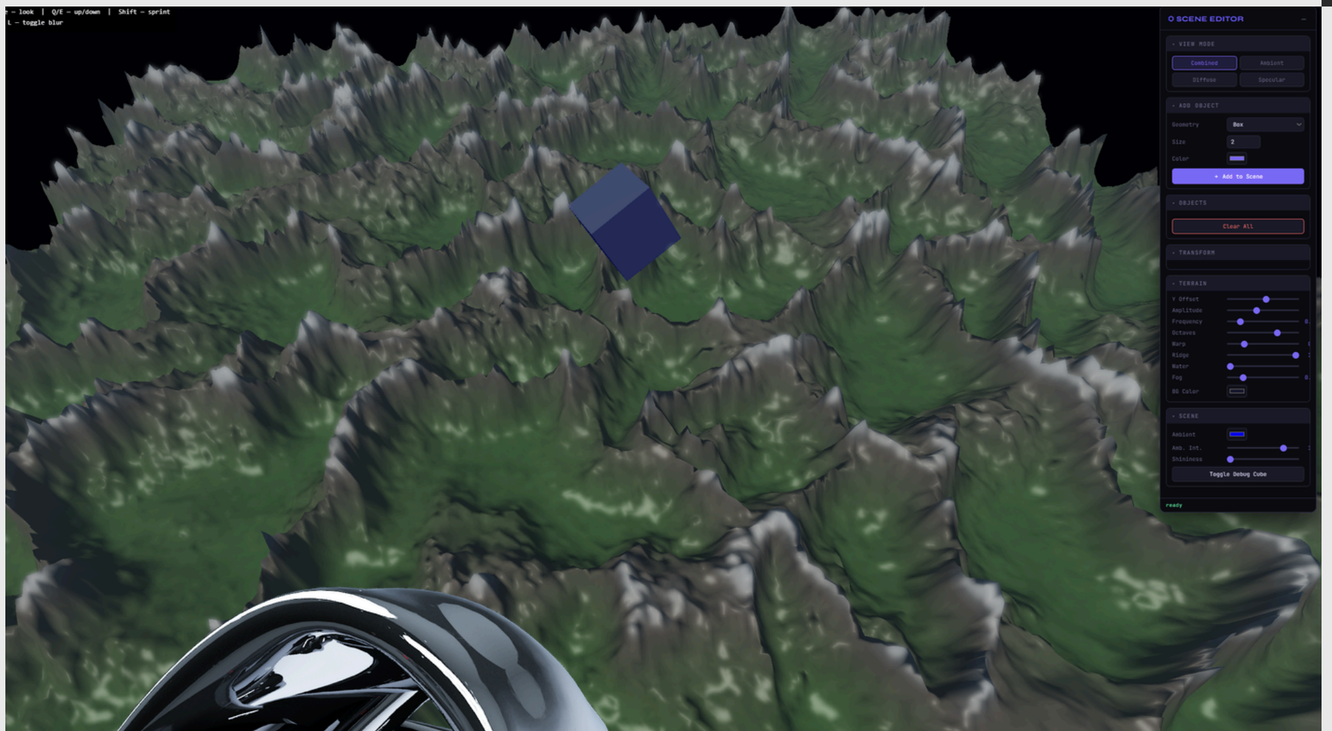
VELOCITY IS SMOOTHED EACH FRAME USING LINEAR INTERPOLATION; FLUID, DAMPED MOVEMENT WITH NO SNAPPING

A FALLBACK ORBIT CAMERA IS AVAILABLE WHEN POINTER LOCK IS UNAVAILABLE



- **A drop-in GUI panel** that appears fixed in the top-right corner, no extra install
- **6 collapsible sections**: View Mode, Add Object, Object List, Transform, Terrain, Scene
- **View Mode buttons** replace the 1/2/3/4 keyboard shortcuts — same functionality, more accessible
- **Add Object** lets you spawn Box/Sphere/Cylinder/Torus/Plane with custom size and colour, using the same Phong material as the rest of the scenE

# SCENE EDITOR




- **Transform section** gives XYZ position/rotation/scale inputs + wireframe toggle on the selected object
- **Terrain section** exposes every noise parameter (amplitude, frequency, octaves, warp, ridge, water blend, fog) as live sliders — great to demo by dragging them in real time during your presentation
- **Scene section** controls ambient light colour + intensity

# SHADER FILES

KEY SHADERS IN THE PROJECT:

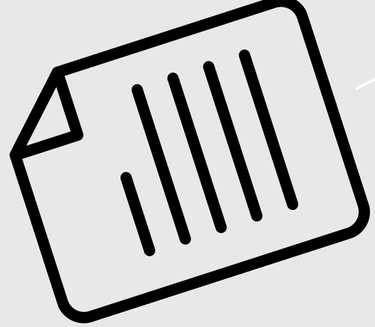
- common/uniforms.glsl — shared uniforms: time, resolution, light & camera position
- common/lighting.glsl — reusable Blinn-Phong ambient, diffuse, specular functions
- noise/simplex.glsl — simplex noise, fBm, ridge fBm, domain warp
- terrain/vertex.glsl — height computation, vertex displacement, finite-difference normals
- terrain/fragment.glsl — color ramp, micro-detail, fog, animated water specular
- post/blur.glsl & post/bloom.glsl — Gaussian blur and bloom reference shaders

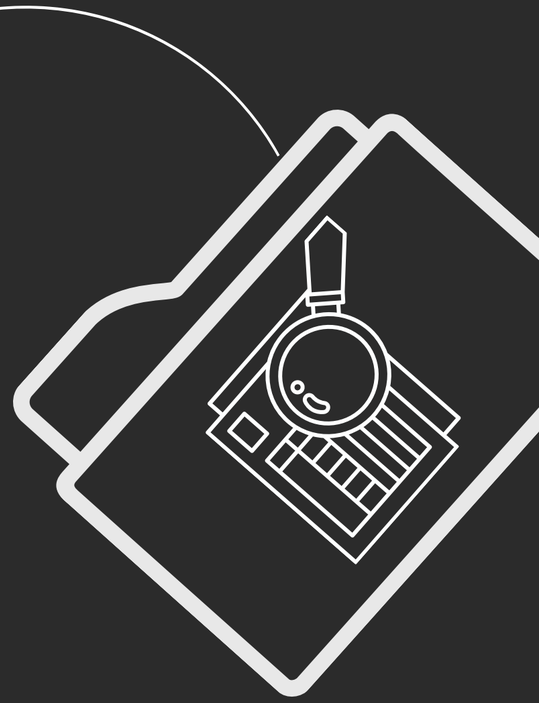


# TESTING

ALL TESTS USE VITEST, RUNNING IN NODE.JS WITH THREE.JS MOCKED;NO BROWSER OR GPU REQUIRED

| TEST FILE | WHAT IT CHECKS |
|---|---|
| Camera.test.js | Camera wrapping and resize response |
| CameraController.test.js | Key bindings, velocity updates, listener cleanup |
| BlurPass.test.js | Render target creation and setSize |
| BloomPass.test.js | Extraction targets and enabled/bypass flag |
| EffectComposer.test.js | Pass ordering and execution sequence |
| OrbitControls.test.js | Fallback camera construction and behavior |

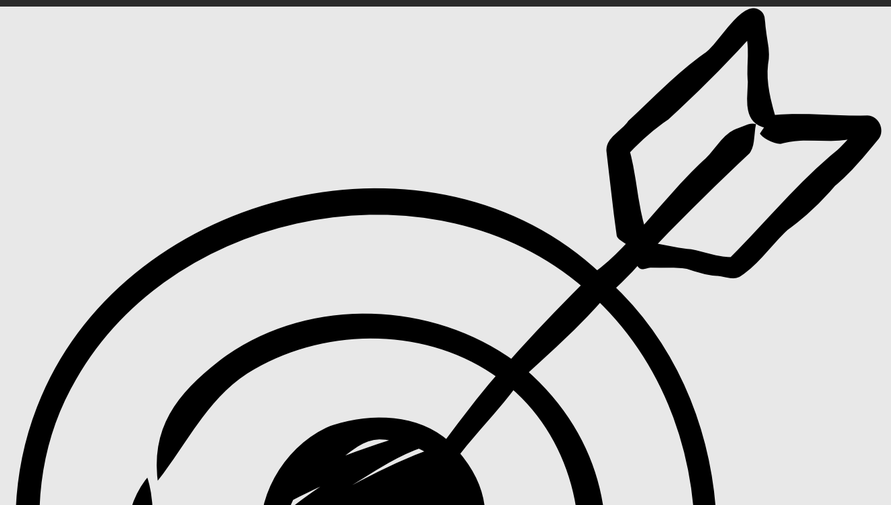






# RESULTS: TERRAIN & LIGHTING

- **Terrain** — fBm with domain warping produces convincing eroded landscapes running smoothly at 60 fps
- **Increasing octaves** from 1 to 6 progressively adds fine detail while preserving large-scale shape
- **Ridge fBm** introduces sharp mountain peaks at minimal extra cost — same noise evaluations reused
- **Lighting** — Blinn-Phong produces correct diffuse gradients and tight specular highlights
- **The debug mode** (isolating ambient / diffuse / specular) was essential for catching shader issues during development



# RESULTS: POST-PROCESSING & PERFORMANCE

- **Bloom** makes bright surfaces like water specular highlights visually glow
- **Reinhard tone mapping** keeps the brightest areas from clipping to pure white
- A key **bug** was caught and fixed: the bloom blur pass was writing to the screen instead of an intermediate buffer, causing the overlay to appear frozen — fixed by allocating a dedicated render target
- **Performance** — ~66,000 vertex shader invocations per frame at 256×256 terrain resolution, all running at 60 fps in Chrome on typical hardware



# LIMITATIONS





- **Vertex-level normals** — normals interpolated across triangles cause visible faceting at low resolutions; per-fragment normals would be more accurate but costlier
- **No shadow casting** — all surfaces receive full lighting regardless of occlusion; adding shadows requires a depth-map render pass from the light's perspective
- **Single light source** — extending to multiple lights needs a uniform array or a deferred shading approach
- **Global bloom** — applies to any bright pixel, not just emissive surfaces; a separate emissive render target would be more physically accurate
- **Editor state not serialized** — Changes made through the scene editor are held in memory only. Refreshing the page resets all uniform values to their coded defaults

# FUTURE IMPROVEMENTS



- **npm package** — publish the renderer as an installable package so users can just run npm install instead of cloning the repo and copying files manually
- **Shadow mapping** — add a depth-map render pass from the light's perspective to support real shadow casting
- **Multiple light sources** — extend the Phong model to support arrays of lights for richer scenes
- **Per-fragment normals** — compute normals inside the fragment shader instead of interpolating from vertices, eliminating faceting at low terrain resolutions
- **More built-in shaders** — expand the prebuilt shader library so users have more starting points beyond terrain and Phong
- **Mobile support** — adapt the camera controls for touch input since Pointer Lock is desktop-only

# CONCLUSION

A COMPLETE REAL-TIME 3D RENDERING SYSTEM BUILT FROM FIRST PRINCIPLES, ENTIRELY IN THE BROWSER

CUSTOM GLSL HANDLES EVERYTHING: PROCEDURAL TERRAIN, PHYSICALLY-INSPIRED LIGHTING, AND POST-PROCESSING EFFECTS

THE MODULAR ARCHITECTURE MEANS NEW SHADERS AND EFFECTS CAN BE ADDED WITHOUT TOUCHING UNRELATED CODE

FBM NOISE, DOMAIN WARPING, AND HEIGHT-BASED COLORING TOGETHER PRODUCE CONVINCING NATURAL-LOOKING TERRAIN

BLOOM AND BLINN-PHONG BRING THE SCENE TO LIFE WITH EXPRESSIVE, DYNAMIC LIGHTING

A LIVE GUI EDITOR MAKES EVERY SHADER PARAMETER TWEAKABLE AT RUNTIME, WITH NO CODE CHANGES NEEDED.

# THANK YOU